EXPERIMENT NO. 4

DESIGN AND IMPLEMENTATION OF EDGE–CLOUD SYNCHRONIZATION USING LOCAL MQTT BRIDGE (GOOGLE COLAB VERSION)

ABISHEK DEVARAJ (192512126)

Aim

To simulate Edge–Cloud synchronization by collecting IoT sensor data at an edge device, synchronizing it to the cloud, and performing real-time analytics using Python in Google Colab.

Objectives

- Simulate an Edge Device.
- Generate real-time IoT sensor data.
- Synchronize data from Edge to Cloud.
- Store synchronized cloud data.
- Perform stream analytics.
- Detect abnormal conditions.
- Visualize synchronized data.

Software required

- Google Colab
- Python 3
- pandas
- numpy
- matplotlib

Theory

- Edge Computing processes data close to the IoT device, reducing latency.
- Cloud Computing stores and analyzes synchronized data for long-term monitoring.

Data Flow:

IoT Device

↓

Edge Device

↓

MQTT Bridge (Simulation)

↓

Cloud Storage

↓

Dashboard

Scenario

- Smart Water Tank Monitoring
- Edge Device collects water level.
- Cloud stores synchronized values.
- Alert generated if water level is very low.

Algorithm

- Start Program.
- Generate Water Tank Level.
- Store data in Edge Memory.
- Synchronize data to Cloud.
- Calculate Mean.
- Calculate Moving Average.
- Calculate Standard Deviation.
- Detect Low Water Level.
- Display Graph.
- Save CSV.

Google colab code

```
# ============================================================
# EXPERIMENT 4
# EDGE-CLOUD SYNCHRONIZATION USING LOCAL MQTT BRIDGE
# GOOGLE COLAB SIMULATION
# SCENARIO: SMART WATER TANK MONITORING
# ============================================================

import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import clear_output
import time


# ============================================================
# EDGE AND CLOUD STORAGE
# ============================================================

edge_storage = []
cloud_storage = []

sample_numbers = []
sync_status = []


# ============================================================
# SYSTEM CONFIGURATION
# ============================================================

total_samples = 60
```

```
critical_level = 25

moving_window = 5


# ============================================================
# REAL-TIME EDGE-CLOUD PROCESSING
# ============================================================

for i in range(1, total_samples + 1):

    # ------------------------------------------------------
    # Generate different water-level samples
    # Every 8th sample is intentionally low
    # ------------------------------------------------------

    if i % 8 == 0:

        water_level = random.randint(12, 24)

    else:

        water_level = random.randint(35, 95)


    # ============================================================
    # EDGE DEVICE
    # ============================================================

    edge_storage.append(water_level)

    sample_numbers.append(i)


    # ============================================================
    # MQTT BRIDGE SIMULATION
    # ============================================================

    # Synchronize edge data with cloud

    cloud_storage.append(water_level)

    sync_status.append("Synchronized")


    # ============================================================
    # CLOUD ANALYTICS
    # ============================================================

    mean_level = np.mean(cloud_storage)

    standard_deviation = np.std(cloud_storage)


    # ------------------------------------------------------
    # Moving Average
    # ------------------------------------------------------

    if len(cloud_storage) >= moving_window:
```

```
    moving_average = np.mean(
        cloud_storage[-moving_window:]
    )

else:

    moving_average = mean_level


# ============================================================
# LOW WATER LEVEL DETECTION
# ============================================================

if water_level < critical_level:

    status = "LOW WATER LEVEL"

else:

    status = "NORMAL"


# ============================================================
# CLEAR PREVIOUS OUTPUT
# ============================================================

clear_output(wait=True)


# ============================================================
# EDGE INFORMATION
# ============================================================

print("=" * 65)

print("EDGE DEVICE - SMART WATER TANK")

print("=" * 65)

print()

print(
    "Current Water Level :",
    water_level,
    "%"
)

print(
    "Edge Records        :",
    len(edge_storage)
)

print(
    "Edge Storage Status :",
    "DATA STORED"
)


# ============================================================
# CLOUD INFORMATION
# ============================================================
```

```
print()

print("=" * 65)

print("CLOUD STORAGE AND ANALYTICS")

print("=" * 65)

print()

print(
    "Cloud Records       :",
    len(cloud_storage)
)

print(
    "Synchronization     :",
    sync_status[-1]
)

print(
    "Mean Water Level    :",
    round(mean_level, 2),
    "%"
)

print(
    "Moving Average      :",
    round(moving_average, 2),
    "%"
)

print(
    "Standard Deviation  :",
    round(standard_deviation, 2)
)

print(
    "Critical Level      :",
    critical_level,
    "%"
)

print(
    "Status              :",
    status
)


# ============================================================
# REAL-TIME GRAPH
# ============================================================

plt.figure(figsize=(11, 5))

plt.plot(
    sample_numbers,
    cloud_storage,
    marker="o",
    label="Cloud Water Level"
```

```
)

plt.axhline(
    critical_level,
    color="red",
    linestyle="--",
    label="Critical Level"
)


# ------------------------------------------------------
# Highlight low water level samples
# ------------------------------------------------------

low_samples = [
    sample_numbers[j]
    for j in range(len(cloud_storage))
    if cloud_storage[j] < critical_level
]

low_values = [
    cloud_storage[j]
    for j in range(len(cloud_storage))
    if cloud_storage[j] < critical_level
]


if len(low_samples) > 0:

    plt.scatter(
        low_samples,
        low_values,
        color="red",
        s=80,
        label="Low Water Alert"
    )


# ------------------------------------------------------
# Graph Labels
# ------------------------------------------------------

plt.xlabel(
    "Sample Number"
)

plt.ylabel(
    "Water Level (%)"
)

plt.title(
    "Edge–Cloud Water Tank Synchronization"
)

plt.grid(True)

plt.legend()

plt.tight_layout()
```

```
    plt.show()


    # ============================================================
    # DELAY FOR REAL-TIME SIMULATION
    # ============================================================

    time.sleep(0.5)


# ============================================================
# CREATE FINAL CLOUD DATAFRAME
# ============================================================

df = pd.DataFrame({

    "Sample": sample_numbers,

    "Water_Level": cloud_storage,

    "Synchronization": sync_status

})


# ============================================================
# ADD STATUS COLUMN
# ============================================================

df["Status"] = df["Water_Level"].apply(

    lambda x:
    "LOW WATER LEVEL"
    if x < critical_level
    else "NORMAL"

)


# ============================================================
# SAVE CLOUD DATA
# ============================================================

file_name = "edge_cloud_water_data_04.csv"

df.to_csv(
    file_name,
    index=False
)


# ============================================================
# FINAL ANALYTICS
# ============================================================

final_mean = df["Water_Level"].mean()

final_std = df["Water_Level"].std()
```

```
final_moving_average = (
    df[″Water_Level″]
    .tail(moving_window)
    .mean()
)

low_water_count = (
    df[″Status″] == "LOW WATER LEVEL"
).sum()


# ============================================================
# FINAL OUTPUT
# ============================================================

print("\n")

print("=" * 65)

print("FINAL EDGE-CLOUD ANALYTICS")

print("=" * 65)

print()

print(
    "Total Samples       :",
    len(df)
)

print(
    "Mean Water Level    :",
    round(final_mean, 2),
    "%"
)

print(
    "Moving Average      :",
    round(final_moving_average, 2),
    "%"
)

print(
    "Standard Deviation  :",
    round(final_std, 2)
)

print(
    "Critical Level      :",
    critical_level,
    "%"
)

print(
    "Low Level Alerts    :",
    low_water_count
)

print(
    "Cloud Records       :",
```

```
    len(cloud_storage)
)

print(
    "CSV File            :",
    file_name
)

print()

print(
    "Experiment Completed Successfully"
)
```

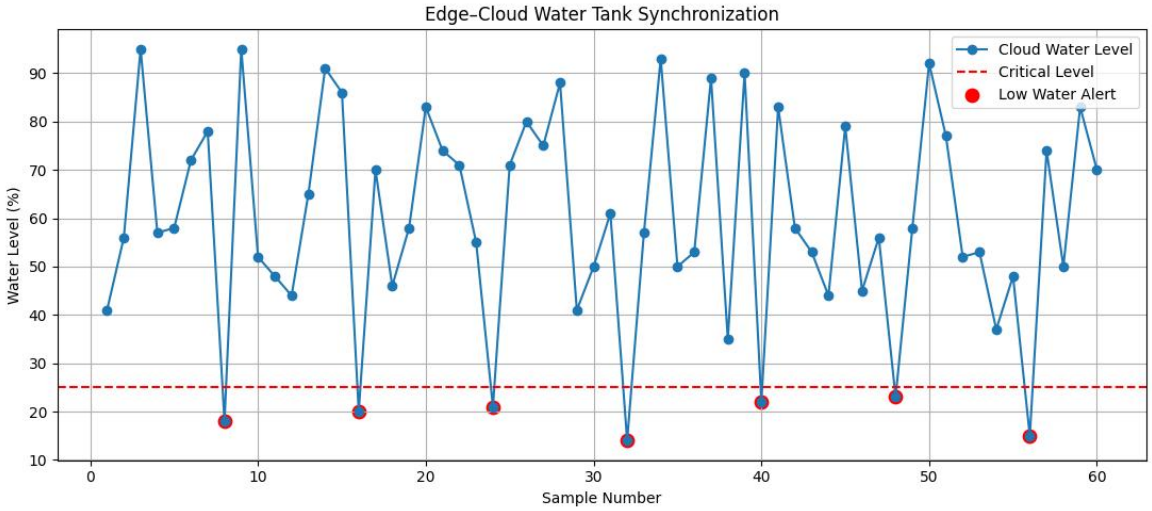
SAMPLE OUTPUT

The following is an **example only**. Your actual Google Colab output will be different because the water-level samples are randomly generated.

```
============================================================
EDGE DEVICE - SMART WATER TANK
============================================================

Current Water Level : 19 %
Edge Records        : 40
Edge Storage Status : DATA STORED

============================================================
CLOUD STORAGE AND ANALYTICS
============================================================

Cloud Records       : 40
Synchronization     : Synchronized
Mean Water Level    : 57.63 %
Moving Average      : 48.40 %
Standard Deviation  : 25.17
Critical Level      : 25 %
Status              : LOW WATER LEVEL
```

At the end, the program produces a final summary similar to:

```
============================================================
FINAL EDGE-CLOUD ANALYTICS
============================================================

Total Samples       : 60
Mean Water Level    : 58.42 %
Moving Average      : 51.80 %
Standard Deviation  : 24.73
Critical Level      : 25 %
Low Level Alerts    : 7
Cloud Records       : 60
CSV File            : edge_cloud_water_data_04.csv
```



Result

The edge device successfully collected water level data and synchronized it with the cloud. The cloud performed stream analytics using Mean, Moving Average, and Standard Deviation, detected low water level conditions, and stored the synchronized data for future analysis.